

并行整体刚度矩阵组装项目长期知识与进展手册

Parallel Global Stiffness Assembly: background, implementation, benchmark evidence, and roadmap

Jiang Haohua

Living deck, first maintained version: 2026-05-15

如何使用这份长期 deck

- 当成个人项目手册：先读背景，再看实现，再看结果证据。
- 当成 mentor 问题索引：遇到 `symbolic assembly`、`scatter plan`、`section/closure` 等概念时，先回到对应章节。
- 当成进展账本：短期 beamer 只服务一次汇报，长期 beamer 保存稳定知识和可追溯结论。
- 当成复现入口：每张结果图和关键数字都应能追溯到仓库文档、结果包或外部文献。

个人复习结论

这份材料优先服务“我自己真正理解项目”，不是为了压缩成 7 分钟汇报。

全篇结构

- 1 项目定位
- 2 FEM 与全局刚度矩阵基础
- 3 稀疏矩阵、CSR 与二阶段组装
- 4 当前 C++ 主线结构
- 5 CPU 并行组装算法
- 6 已有 benchmark 证据
- 7 平台差异、进度与路线
- 8 维护规则

项目一句话定位

研究对象

在共享内存多核 CPU 上，研究如何把大量有限元单元刚度矩阵高效、正确地累加到全局稀疏刚度矩阵。

不是当前主线

- 继续扩展 GPU 算法。
- 做完整商业求解器。
- 优化线性方程组求解器。

当前主线

- CPU 并行组装算法对比。
- 真实工程网格可复现 benchmark。
- 结果、图表、文档和短期汇报可追溯。

Source: README.md; docs/requirements/cpu-parallel-stiffness-assembly-design.md

为什么 assembly 值得单独研究

- 有限元里每个单元都能独立计算局部矩阵，看起来天然并行。
- 但多个单元会共享节点和自由度，写回全局矩阵时会写到同一位置。
- 因此并行 assembly 的难点不是“算 K_e ”，而是[如何避免或管理写冲突](#)。
- 真实大网格上，全局矩阵很稀疏，不能用 dense matrix 方式处理。

个人复习结论

并行 assembly 是“局部计算容易、全局写入困难”的问题。

Source: docs/requirements/cpu-parallel-stiffness-assembly-design.md; ScienceDirect S0045782524003323

当前唯一主线目录

继续开发入口

`parallel_global_stiffness_assembly/cpu_parallel_stiffness_assembly`

- 仓库根 README 已明确：当前项目聚焦 CPU 平台并行整体刚度矩阵组装。
- GPU 历史内容保留为 legacy 参考，不是当前默认入口。
- 短期 beamer 已在 `reports/2026-05-14-mentor-next-steps-beamer/`。
- 本长期 beamer 固定放在 `reports/project-long-term-beamer/`。

个人复习结论

后续新增知识、稳定结果和进展复盘，合并到长期 deck；日期目录只承载一次汇报。

项目最终要回答的问题

- ① 图着色法在多核 CPU 上是否仍然是合适基线？
- ② 直接累加和 atomic 在真实工程网格上是否可接受？
- ③ 线程私有缓冲、COO sort-reduce、row-owner 等路线各自代价是什么？
- ④ 规则网格和真实 C3D4 工程网格上的趋势是否一致？
- ⑤ 工程上后续应该优先保留哪类算法继续优化？

Source: docs/requirements/cpu-parallel-stiffness-assembly-design.md

当前已经具备的基础

实现基础

- 3D Tet4 / Hex8。
- 每节点 3 个位移 DOF。
- CSR 稀疏结构预计算。
- AssemblyPlan 和 scatter plan。
- Abaqus .inp 基础解析。

Source: [parallel_global_stiffness_assembly/cpu_parallel_stiffness_assembly/README.md](#)

实验基础

- 串行 correctness baseline。
- 多个 CPU 并行 assembler。
- CSV / JSON / Markdown 输出。
- 结果图表和跨平台 schema。

真实工程网格为什么重要

- 3d-WindTurbineHub.inp 是当前核心真实算例。
- 规模约为 **228384** nodes、**1113684** elements、**685152** DOFs。
- 单元类型为 Tet4 / Abaqus C3D4。
- 真实非结构化网格更能暴露写冲突、稀疏结构构建、内存和缓存行为。

个人复习结论

小规则网格适合 smoke test；WindTurbineHub 才是工程可用性判断的主要证据。

Source: [results/2026-05-11-symbolic-numeric/symbolic_numeric_eval_report.md](#)

短期 beamer 和长期 beamer 的边界

类型	目的	更新原则
短期 beamer	服务一次 mentor/组会/课程汇报	围绕日期和听众压缩叙事，可以只选部分证据
长期 beamer	服务个人掌握完整项目	保存稳定概念、结果、来源、进度和待办
结果报告	保存实验输出和图表	不为演讲改事实，只按数据生成

个人复习结论

长期 deck 不是把短期 deck 堆起来，而是吸收其中经验证的稳定知识。

从结构问题到矩阵方程

有限元离散后的典型形式

$$Ku = f$$

- K : global stiffness matrix, 描述整体结构刚度。
- u : 全局自由度位移向量。
- f : 外载荷、边界条件处理后的右端项。
- assembly 的目标: 把每个单元的局部刚度矩阵 K_e 累加到 K 。

Source: standard FEM formulation; docs/requirements/cpu-parallel-stiffness-assembly-design.md

什么是 element stiffness matrix

局部视角

- 每个 element 有自己的节点。
- 每个节点有若干 DOF。
- 单元内部形成小矩阵 K_e 。

全局视角

- 不同 element 共享全局节点。
- 局部 DOF 要映射到全局 DOF。
- 多个 K_e 贡献到同一个 K 。

个人复习结论

K_e 是局部刚度， K 是所有局部刚度按连接关系累加后的整体刚度。

DOF: 自由度是矩阵维度的来源

- DOF 是 degree of freedom, 即节点上未知量的编号。
- 当前 C++ 主线默认每节点 **3** 个位移自由度: u_x, u_y, u_z 。
- 如果节点数是 N , 全局位移向量长度约为 $3N$ 。
- WindTurbineHub 约 **228384** nodes, 因此 DOFs 约 **685152**。

Source: README.md; results/2026-05-11-symbolic-numeric/symbolic_numeric_eval_report.md

Tet4、Hex8 与 Abaqus C3D4

术语	含义	当前项目角色
Tet4	4 节点四面体单元	当前真实工程算例主单元
Hex8	8 节点六面体单元	支持规则网格实验，但不是当前物理主线
C3D4	Abaqus 里的 3D 4-node tetrahedral element	3d-WindTurbineHub.inp 中的真实输入类型

个人复习结论

Tet4 是代码单元类型，C3D4 是 Abaqus 输入文件中的对应名字。

mesh topology 与 kernel mode 是两条轴

mesh topology

- 单元有几个节点。
- 节点如何连接成 element。
- 决定 DOF 映射和稀疏 pattern。

kernel mode

- 局部矩阵 K_e 怎么计算。
- simplified: 确定性合成矩阵。
- physics_tet4: 使用 Tet4 物理刚度公式。

个人复习结论

不要把“网格是什么形状”和“局部刚度怎么算”混为一谈。

Source: reports/2026-05-14-mentor-next-steps-beamer/mentor_next_steps_beamer.tex

simplified kernel 有什么用

- 它不追求真实材料和几何物理。
- 它生成确定性的局部刚度矩阵，便于隔离 assembly strategy 的差异。
- 它适合快速看写冲突、scatter、CSR 写入和线程扩展趋势。
- 它不能替代真实 Tet4 物理 benchmark。

个人复习结论

simplified 是 synthetic FEM assembly benchmark，不是完整真实 FEM。

Source: docs/cpu/cpu_algorithms.md; prior kernel-mode clarification

physics_tet4 做了什么

- 从四面体节点坐标计算体积。
- 计算形函数梯度和应变-位移矩阵 B 。
- 使用材料矩阵 D 。
- 形成局部刚度：

$$K_e = B^T D B \cdot V$$

- 然后把 K_e 按 scatter 位置累加到全局 CSR values。

Source: `src/assembly/element_kernels.cpp`; `reports/2026-05-14-mentor-next-steps-beamer`

为什么全局刚度矩阵是稀疏矩阵

- 每个单元只连接少数节点。
- 一个 DOF 只和相邻单元中的 DOF 有直接刚度耦合。
- 绝大多数远距离 DOF 之间没有直接矩阵条目。
- 因此 K 的非零项远少于 dense matrix 的 n^2 项。

个人复习结论

稀疏性来自有限元网格的局部连接关系。

CSR: 当前项目的全局矩阵格式

Compressed Sparse Row

CSR 用三类数组描述稀疏矩阵：每行起止位置、列索引、非零值。

- `row_ptr`: 第 i 行非零项在数组中的起止位置。
- `col_idx`: 每个非零项所在列。
- `values`: 每个非零项的数值。
- symbolic 阶段确定 `row_ptr` 和 `col_idx`; numeric 阶段更新 `values`。

个人复习结论

CSR 把“结构”和“数值”天然分开，正好对应 symbolic/numeric split。

local-to-global mapping

- 每个 element 有局部自由度编号: $0, 1, \dots, n_e - 1$ 。
- 每个局部自由度对应一个全局 DOF。
- $K_e(i, j)$ 需要加到 $K(g_i, g_j)$ 。
- 这里的 g_i, g_j 来自 element connectivity 和 DOF 规则。

个人复习结论

assembly 的基本动作就是：找全局位置，然后累加。

scatter-add 是什么

scatter-add

把局部矩阵条目分散写入全局稀疏矩阵的指定位置，并与已有贡献累加。

- scatter: 从局部 $K_e(i, j)$ 找到全局 CSR values 下标。
- add: 多个 element 对同一全局位置的贡献需要相加。
- 并行难点: 多个线程可能同时 add 到同一位置。

Source: docs/cpu/cpu_algorithms.md; include/assembly/assembly_plan.h

symbolic assembly: 先确定结构

- 只处理拓扑、DOF 和稀疏 pattern。
- 构建 CSR row_ptr / col_idx。
- 缓存每个单元的全局 DOF 列表。
- 预计算 $K_e(i,j)$ 应写入的 CSR values 位置。
- 不计算真实局部刚度数值。

Source: docs/cpu/symbolic_numeric_assembly.md

numeric assembly: 再填充数值

- 计算每个 element 的 K_e 。
- 复用 symbolic 阶段得到的 DOF 和 scatter 信息。
- 把 K_e 的条目累加到 CSR values。
- 当材料状态、载荷步或迭代状态变化但网格不变时，只需要重复 numeric 阶段。

Source: docs/cpu/symbolic_numeric_assembly.md

无符号直接组装是什么

- 不提前复用 CSR pattern 或 scatter plan。
- 每次从 element DOF 直接生成大量 (row,col,value) 贡献。
- 对贡献排序，把相同 row,col 的值 reduce。
- 最后形成全局稀疏矩阵。

个人复习结论

无符号直接组装不是“没有数学结构”，而是运行时不复用全局稀疏写入位置。

Source: [results/2026-05-11-symbolic-numeric/symbolic_numeric_eval_report.md](#)

mentor 示例与当前 C++ 的对应关系

Mentor 示例概念	当前 C++ 主线	解释
build_symbolic_pattern	CsrMatrix::build_sparsity	用单元 DOF 外积建立 CSR pattern
cellDofsCache	AssemblyPlan::dofs	缓存每个单元的全局 DOF
allocate_global_matrix	CsrMatrix values 清零复用	结构不变，只更新数值
assemble_numeric	各 assembler 的 assemble()	计算 K_e 并写回全局矩阵
无直接对应项	AssemblyPlan::scatter	预先保存 $K_e(i, j)$ 到 CSR value 的位置
section/closure	固定每节点 3 DOF	首阶段不引入通用拓扑抽象

Source: docs/cpu/symbolic_numeric_assembly.md

什么是 section

- 在 PETSc/DMPlex 语境里，PetscSection 描述 mesh point 上挂了多少 DOF，以及这些 DOF 在向量中的 offset。
- 它回答的问题是：某个节点、边、面、单元上有哪些未知量？
- 当前项目固定每个节点 3 DOF，因此暂时不需要完整 Section 抽象。

个人复习结论

section 是“网格对象到数据布局”的映射，不是 assembly 算法本身。

Source: PETSc DMPlex documentation: petsc.org/main/manual/dmplex/; `DMPlexCreateSection`

什么是 closure

- closure 指一个 cell 以及构成它的低维拓扑对象集合。
- 对三维四面体来说，closure 可以包含 cell、faces、edges、vertices。
- PETSc 的 `DMPlexVecGetClosure` 可以一次取出某个 cell 相关的局部数据。
- 当前项目只需要节点 DOF，因此直接从 element connectivity 得到节点集合。

Source: PETSc DMPlex manual; [DMPlex: Unstructured Grids in PETSc](#)

为什么当前不重构为 PETSc-style 抽象

- 当前目标是回答 CPU assembly strategy 的效率与正确性。
- 代码已经有可用的 Mesh、DofMap、CsrMatrix、AssemblyPlan。
- 引入通用 Section/Closure 会增加抽象成本，短期不改变 benchmark 结论。
- 如果未来做高阶单元、多物理场、边/面 DOF，再考虑这层抽象。

个人复习结论

先保留当前 C++ 结构，把 mentor 概念解释清楚；不要为了术语一致而过早重构。

Source: docs/cpu/symbolic_numeric_assembly.md

代码架构的四层

- ① 输入与网格层：规则网格、.inp、节点、单元。
- ② 稀疏结构与计划层：CsrMatrix、DofMap、AssemblyPlan。
- ③ 组装算法层：串行、atomic、private CSR、COO、coloring、row-owner。
- ④ benchmark 与结果层：CLI、CSV/JSON/Markdown、图表、跨平台 schema。

Source: README.md; docs/requirements/cpu-parallel-stiffness-assembly-design.md

核心数据结构：Mesh

- 保存节点坐标和单元连接关系。
- 支持规则 cube mesh 和 Abaqus .inp 输入。
- 对 assembly 来说，最重要的是 element 的全局节点列表。

个人复习结论

Mesh 给出 “有哪些 element，以及每个 element 连到哪些 node”。

Source: `include/core/mesh.h`; `src/core/mesh.cpp`

核心数据结构：DofMap

- 把节点编号映射成全局 DOF 编号。
- 当前每个节点固定 3 个位移 DOF。
- 支撑 element-local DOF 到 global DOF 的转换。

个人复习结论

DofMap 是当前项目的轻量 section。

Source: `include/core/dof_map.h`; `docs/cpu/symbolic_numeric_assembly.md`

核心数据结构：CsrMatrix

- 保存全局刚度矩阵的稀疏结构和值。
- `build_sparsity()` 对应 symbolic pattern 构建。
- `values` 可以清零并重复填充。
- correctness check 默认与串行 baseline 比较。

Source: `include/core/csr_matrix.h`; `src/core/csr_matrix.cpp`

核心数据结构：AssemblyPlan

- 保存每个 element 的全局 DOF 列表。
- 保存每个 $K_e(i,j)$ 对应的 CSR values 下标。
- 数值阶段不再做昂贵查找。
- 它是当前 C++ 主线比 mentor 教学示例更工程化的部分。

个人复习结论

`AssemblyPlan::scatter` 是理解当前实现性能边界的关键。

Source: `include/assembly/assembly_plan.h`; `src/assembly/assembly_plan.cpp`

算法统一接口

- IAssembler 定义统一 assemble 接口。
- AssemblerFactory 根据 CLI 名称创建算法实现。
- benchmark 主程序可以用相同输入循环比较多个算法。
- 新增算法不应该重写 benchmark 主流程。

Source: `include/assembly/assembler_interface.h`; `include/assembly/assembler_factory.h`

benchmark CLI 主流程

- ① 读取或生成 mesh。
- ② 构建 DOF map、CSR sparsity、AssemblyPlan。
- ③ 运行串行 baseline。
- ④ 对每个算法和线程数重复 warmup/repeat。
- ⑤ 检查 relative L2、max abs 等正确性指标。
- ⑥ 输出 CSV、JSON、Markdown summary 和图表。

Source: `apps/benchmark/main.cpp`; `scripts/run_cpu_experiments.py`

当前结果字段为什么这么多

- 总时间不足以解释性能来源。
- 需要拆开 preprocess、numeric、merge、sort、reduce、memory 等指标。
- 平台字段记录 CPU、编译器、OpenMP、profile 和 env group。
- 这些字段支撑跨平台比较和短期/长期汇报复用。

Source: [parallel_global_stiffness_assembly/cpu_parallel_stiffness_assembly/README.md](#)

当前比较的算法集合

- `cpu_serial`: 串行基线。
- `cpu_atomic`: OpenMP atomic 直接累加共享 CSR。
- `cpu_private_csr`: 线程私有 values, 最后归并。
- `cpu_coo_sort_reduce`: 生成 COO 贡献, 排序规约。
- `cpu_graph_coloring`: 同色无冲突 element 并行。
- `cpu_row_owner`: owner-computes / 行拥有者原型。

Source: README.md; docs/cpu/cpu_algorithms.md

串行基线: `cpu_serial`

- 单线程遍历 element。
- 计算 K_e 。
- 按 scatter plan 写回 CSR values。
- 作为 correctness 和 speedup 的基线。

个人复习结论

串行基线不一定最快，但它定义了“正确答案”和加速比基准。

Source: docs/cpu/cpu_algorithms.md; src/backends/cpu/serial_assembler.cpp

直接累加: `cpu_atomic`

- 多线程同时遍历 element。
- 对共享 CSR values 执行 atomic add。
- 优点：实现直接，内存额外开销小。
- 缺点：热点写入和同步开销可能限制扩展性。

Source: docs/cpu/cpu_algorithms.md; arXiv:2012.00585

线程私有缓冲: `cpu_private_csr`

- 每个线程维护自己的 CSR values 副本或局部累加区。
- 数值组装阶段减少共享写冲突。
- 最后把各线程结果归并到全局 values。
- 代价: 额外内存和 merge 阶段。

个人复习结论

private CSR 用内存换同步开销。

Source: `docs/cpu/cpu_algorithms.md`

COO 排序规约: `cpu_coo_sort_reduce`

- 先生成所有 (row,col,value) 贡献。
- 按 row,col 排序。
- 对相同位置的贡献 reduce。
- 代价集中在生成、排序、规约和临时内存。

Source: docs/cpu/cpu_algorithms.md; symbolic_numeric_eval_report.md

图着色: `cpu_graph_coloring`

- 把 element 分成多个 color。
- 同一 color 内的 element 不共享写入位置。
- 每个 color 内可以无 atomic 并行组装。
- 代价：着色预处理、颜色数量、颜色间同步、负载均衡。

Source: docs/cpu/cpu_algorithms.md; ScienceDirect S0045782524003323

row-owner: `cpu_row_owner`

- 按全局矩阵行划分 owner。
- 每个线程主要负责自己拥有的行。
- 目标是减少任意线程写任意位置带来的冲突。
- 当前仍应理解为 owner-computes 原型，而不是最终最优实现。

Source: `docs/cpu/cpu_algorithms.md`

算法对比的第一原则

公平比较

不同算法必须共享同一个 mesh、DOF map、CSR structure、AssemblyPlan、kernel、correctness baseline 和统计口径。

- 否则速度差异可能来自输入或数据结构差异。
- 结果必须同时看时间、加速比、效率、内存和正确性。
- 任何平台差异都要写入结果包，而不是混入算法结论。

已有结果证据的几条线

- 2026-04-22: 初期 CPU benchmark 图表和摘要。
- 2026-04-28: 12 charts presentation figures, 区分 correctness / efficiency / memory。
- 2026-05-11: symbolic vs numeric efficiency evaluation。
- 2026-05-11/12/14: thread scaling 与 P/E-core profile。
- results/cross-platform-v1/: 跨平台 schema 和 core-profile comparison。

Source: results/; source_index.md

symbolic reuse vs direct no-symbolic: 主结论

组装次数	symbolic reuse 摊销总耗时 ms	direct no-symbolic 总耗时 ms	收益
1	3818.786	6276.199	1.644x
3	1633.165	5822.369	3.565x
10	901.325	5696.788	6.320x
30	686.745	5552.854	8.086x

个人复习结论

单次组装也受益；多次组装更能体现 symbolic result reuse 的摊销优势。

Source: results/2026-05-11-symbolic-numeric/symbolic_numeric_eval_report.md

为什么多次组装收益更明显

- symbolic 阶段成本主要来自 CSR pattern 和 scatter plan。
- 网格拓扑和 DOF 编号不变时，这部分可以只做一次。
- numeric 阶段随材料状态、迭代状态、时间步重复执行。
- 组装次数越多，symbolic 成本摊到每次上的比例越低。

个人复习结论

多次组装场景下，二阶段组装不是微优化，而是改变成本结构。

控制实验：每次重建 symbolic 结构

组装次数	symbolic 构建次数	摊销总耗时 ms	相对 direct 收益
1	1	3813.269	1.646x
3	3	3735.218	1.559x
10	10	3743.918	1.522x
30	30	3394.580	1.636x

个人复习结论

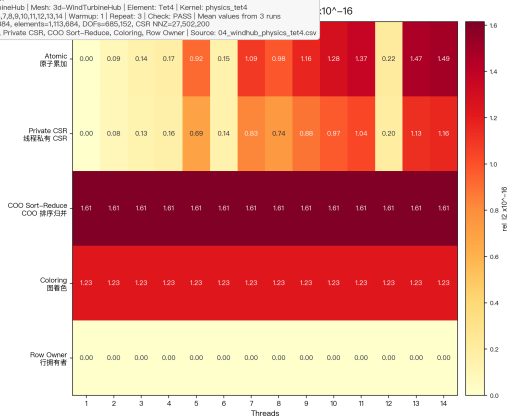
`symbolic_rebuild_serial` 说明：主收益来自符号结果复用，但 CSR/scatter 路径本身也比直接排序归并更合理。

Source: [results/2026-05-11-symbolic-numeric/symbolic_numeric_eval_report.md](#)

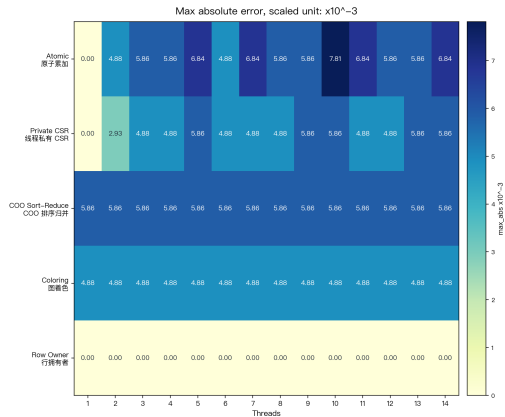
WindHub physics correctness evidence

正确性统计 / Correctness Heatmaps: 3d-WindTurbineHub / physics_tet4

配置 / Configuration
Case: 3d-WindTurbineHub | Mesh: 3d-WindTurbineHub | Element: Tet4 | Kernel: physics_tet4
Threads: 1,2,3,4,5,6,7,8,9,10,11,12,13,14 | Warmup: 1 | Repeat: 3 | Check: PASS | Mean values from 3 runs
Solver: nodes=228,384, elements=1,113,684, DOF=685,152, CSR NNZ=27,502,200
Algorithms: Atomic, Private CSR, COO Sort-Reduce, Coloring, Row Owner | Source: 04_windhub_physics_tet4.csv



All cell values are annotated with two decimals. Unit scaling avoids repeated long scientific notation in every cell. Serial is used only as baseline.



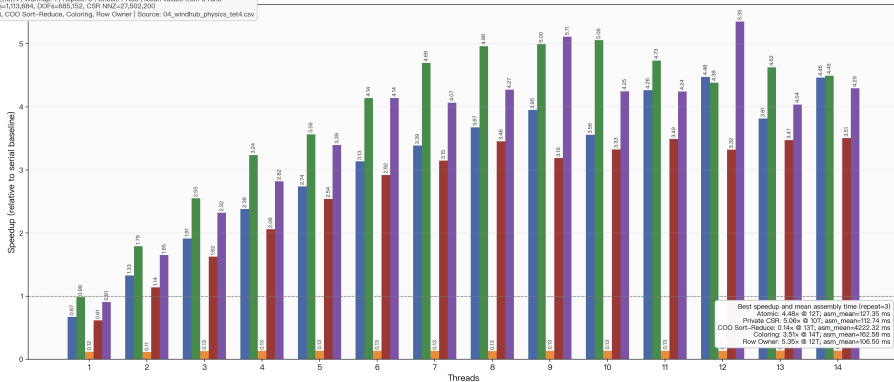
Source: results/2026-04-28-12charts-repeat3-threads1to14/presentation_charts_12_v2

WindHub physics efficiency evidence

效率统计 / Efficiency Grouped Bars: 3d-WindTurbineHub / physics_tet4

配置 / Configuration

Case: 3d-WindTurbineHub | Mesh: 3d-WindTurbineHub | Element: Tet4 | Kernel: physics_tet4
Threads: 1,2,3,4,5,6,7,8,9,10,11,12,13,14 | Warmup: 1 | Repeat: 3 | Check: PASS | Mean values from 3 runs
Scale: nodes=228,384, elements=1,113,684, DOFs=685,152, CSR NNZ=27,502,200
Algorithms: Atomic, Private CSR, COO Sort-Reduce, Coloring, Row Owner | Source: 04_windhub_physics_tet4.csv



Grouped bars replace line-only presentation. Every bar is annotated with speedup to two decimals; mean assembly time is valid because repeat=3.

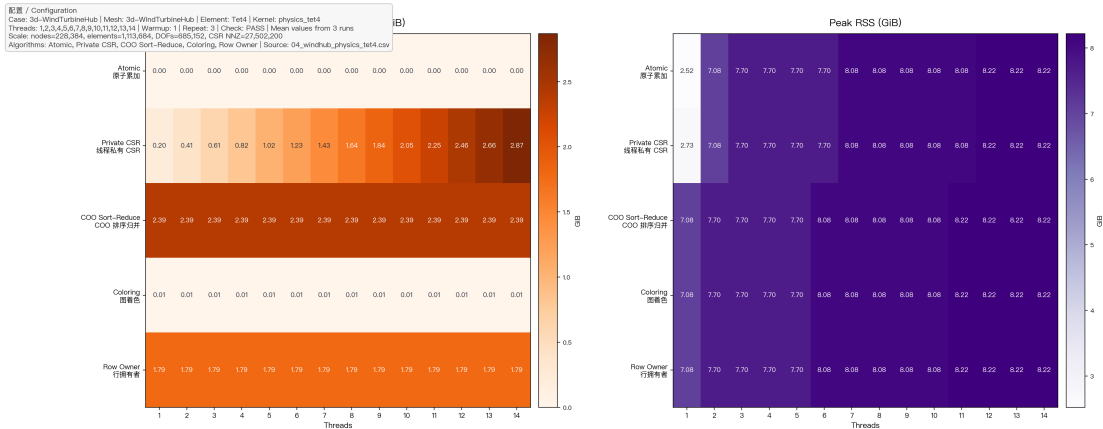
Atomic Private CSR COO Sort-Reduce Coloring Row Owner

Source:

results/2026-04-28-12charts-repeat3-threads1to14/presentation_charts_12_v2

WindHub physics memory evidence

内存占用统计 / Memory Heatmaps: 3d-WindTurbineHub / physics_tet4



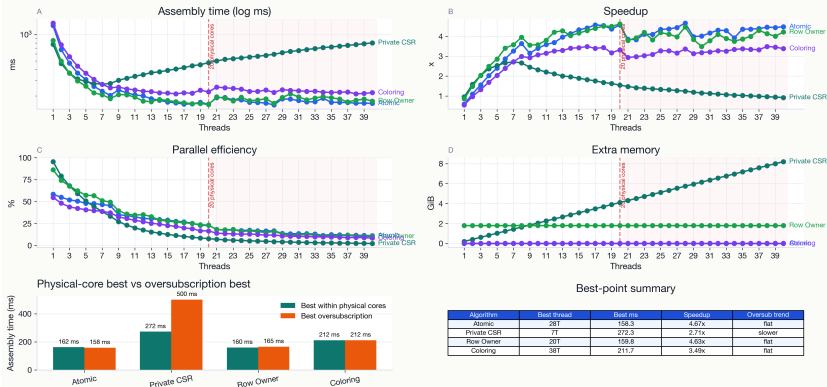
All memory values are annotated with two decimals. Extra memory is benchmark-estimated; Peak RSS is process resident memory.

Source: results/2026-04-28-12charts-repeat3-threads1to14/presentation_charts_12_v2

Intel full-host thread scaling

Thread Scaling Dashboard: bound

Large panels show scaling behavior; the lower panels summarize the fastest assembly points.



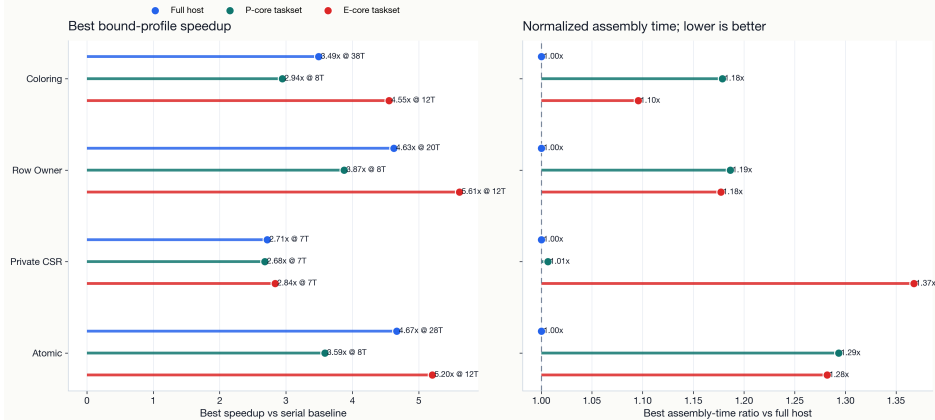
Source:

results/2026-05-11-thread-scaling-linux-intel/figures

Intel P-core-only 与 E-core-only

Intel Core Ultra 7 265KF Core-Profile Acceleration Comparison

full host vs P-core-only vs E-core-only; Linux taskset affinity-restricted profiles



Source:

results/cross-platform-v1/figures; results/2026-05-12-thread-scaling-linux-intel-hybrid-core-supplement.md

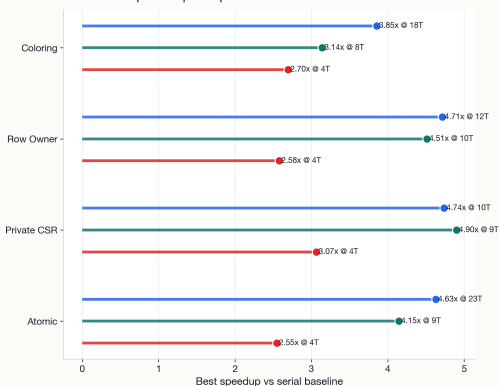
Apple M4 Max QoS-biased profile

Apple M4 Max Core-Profile Acceleration Comparison

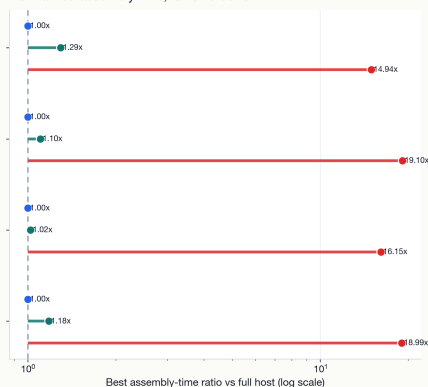
full host vs Performance QoS vs Efficiency QoS; QoS-biased, not hard-pinned core affinity

● Full host ● Performance QoS ● Efficiency QoS

Best bound-profile speedup



Normalized assembly time; lower is better



Source:

[results/cross-platform-v1/figures](#); [results/2026-05-14-thread-scaling-macos-m4max-qos-supplement.md](#)

benchmark figures 与 presentation charts 的区别

类型	目标	约束
benchmark/report figures presentation charts	保存真实结果和可追溯性 帮助听众快速理解结论	不为了好看改变数据口径 可以重新排版，但必须标注来源 和边界
长期 beamer figures	个人复习和进度追踪	优先可追溯，必要时吸收 presentation 图

个人复习结论

图表可以美化，benchmark truth 不能被美化改写。

为什么要区分平台差异

- CPU 架构、核心类型、调度机制、编译器和 OpenMP runtime 都会影响结果。
- Apple M4 Max 与 Intel Core Ultra 7 265KF 的 P/E-core profile 机制不同。
- benchmark 结论必须写清楚平台字段和 run profile。
- 跨平台比较用于解释趋势，不直接把绝对时间写成算法本质优劣。

Source: docs/platform/cross-platform-benchmark-schema.md; results/cross-platform-v1/cross_platform_schema_report.md

Intel 的 taskset profile

- `full_host`: 使用整机可用 CPU。
- `performance_core_only`: 通过 `taskset -c 0-7` 限制到 P-core 集合。
- `efficiency_core_only`: 通过 `taskset -c 8-19` 限制到 E-core 集合。
- CSV 中 `host physical/logical core` 字段仍描述整机；有效限制来自外层 `taskset`。

Source: [results/2026-05-12-thread-scaling-linux-intel-hybrid-core-supplement.md](#)

Apple 的 QoS-biased profile

- macOS 没有与 Linux `taskset` 等价的公开硬绑核接口。
- `performance_core_only` 和 `efficiency_core_only` 在 Apple 结果中是 QoS-biased sensitivity run。
- Performance QoS: 正常前台/default scheduling。
- Efficiency QoS: `taskpolicy -c background`。

个人复习结论

Apple profile 是调度偏置实验，不是硬件隔离实验。

Source: [results/2026-05-14-thread-scaling-macos-m4max-qos-supplement.md](#)

当前已完成的稳定进展

- CPU 主线目录和 README 已明确。
- 多算法 assembler 框架已经具备。
- 真实 WindHub 网格 benchmark 已有多轮结果。
- symbolic/numeric split 已经文档化并完成效率评估。
- Intel 与 Apple 的 core-profile 图表和 supplement 已形成。
- 已有短期 mentor beamer，可作为阶段性汇报材料。

Source: README.md; docs/cpu; results/; reports/2026-05-14-mentor-next-steps-beamer

当前限制

- 当前仍聚焦 assembly, 不覆盖完整求解器 pipeline。
- Section/Closure 还没有实现为通用抽象。
- Apple QoS profile 不能等同于硬绑核 P/E-core 测试。
- benchmark 结果仍需要持续区分真实结果图和 presentation 图。
- 外部文献引用目前只作为背景锚点, 尚未形成完整 literature review。

下一步路线：研究层

- ① 固定 symbolic/numeric 术语和 benchmark 口径。
- ② 继续比较不同算法在真实工程网格上的效率、内存和正确性。
- ③ 增强跨平台 benchmark package 的可读性和可复现性。
- ④ 如果 mentor 继续追问通用 FEM 框架，再评估 Section/Closure 重构价值。

下一步路线：文档与汇报层

- ① 每次短期 beamer 结束后，把稳定结论合并回长期 deck。
- ② 每张长期 deck 中引用的结果图都登记在 `source_index.md`。
- ③ mentor 新概念先放入长期 deck 的概念解释区，再决定是否改代码。
- ④ 对外汇报使用短期 deck；个人复习使用长期 deck。

个人复习结论

长期 deck 是项目知识的“主账本”。

长期 deck 的更新规则

- ① 新增稳定概念：加到对应章节，并写 notes。
- ② 新增结果图：直接引用 `results/` 路径，并更新 `source_index.md`。
- ③ 新增短期汇报：只把稳定结论合并进长期 deck，不整段复制短期叙事。
- ④ 新增外部文献：说明它支持哪个概念，不把长期 deck 变成无边界文献综述。
- ⑤ 修改平台解释：同时检查所有相关图表页和 notes。

结果图引用策略

当前决策

长期 deck 不复制结果图到本目录，直接引用 ../../results/... 中的 PNG。

- 好处：结果图更新后，长期 deck 可以同步反映最新图表。
- 风险：如果清理或移动 results 目录，编译会失败。
- 缓解：source_index.md 记录所有引用路径；验证脚本/命令检查路径存在。

推荐维护检查

- 检查 TeX 输入：所有 `\input{}` 文件存在。
- 检查图片路径：所有 `\includegraphics{}` 指向存在的 PNG。
- 检查来源清单：每个结果图和外部链接都在 `source_index.md`。
- 检查编译：优先 XeLaTeX；本机可尝试 Tectonic。
- 检查语言：中文解释，英文术语和代码名保留。

外部资料只做概念锚点

- PETSc/DMPlex: 解释 section/closure 和 mesh-data layout。
- shared-memory FEM assembly papers: 解释 coloring、atomic、race condition、sparse assembly 背景。
- 项目事实仍以仓库代码、docs、results 为准。

个人复习结论

外部文献帮助理解术语；项目结论必须回到本仓库证据。

最后保留的主线判断

- ❶ 项目当前主线是 **共享内存 CPU 上的整体刚度矩阵并行组装**，不是 GPU 新算法扩展。
- ❷ 真实比较必须共享同一个 **Mesh / CSR / AssemblyPlan / correctness baseline**。
- ❸ **symbolic assembly** 的价值不只是术语，而是把拓扑、DOF、CSR pattern 和 scatter 写入位置提前固定。
- ❹ 多次组装场景下，符号结果复用会显著摊销成本；单次组装也受益于避免全局贡献排序归并。
- ❺ Intel 的 taskset P/E-core 实验和 Apple 的 QoS-biased profile 机制不同，不能混写。